

VA: Graphical interfaces and event-driven programming

- This assignment is not mandatory, but can give extra points during the exam, replacing one of the B section exercises. Extra points are given for this course instance only (exam and first offered re-exam).
- This voluntary assignment concerns writing an event-driven program with either a graphical user interface (GUI) for a physical object simulation, or a web server for the calculator.
- To present the VA, describe your code, design choices, and demonstrate its functionality.
- When the assignment has been approved you should upload the code to STUDIUM.

Tip: If your GUI program is a game, i.e. a TA can move an object, it is easier to pass the assignment. Especially, if it is a game for two e.g. table hockey with goalposts.

Important note:

You may collaborate with other students, search the Internet, and get a general overview using AI tools, but you must write and be able to explain the code you actually present. You may not copy code, either from other students, automated tools, or from the Internet, except from those places explicitly pointed out in this module.

Changing variable names and similar superficial modifications does not count as writing your own code. Also, aiding somebody to cheat (for example, if somebody hands in your code as their own) is not permitted and will be reported.

Since these assignments are included as part of the course examination, we are obliged to report blatant violations of these rules to the disciplinary board.

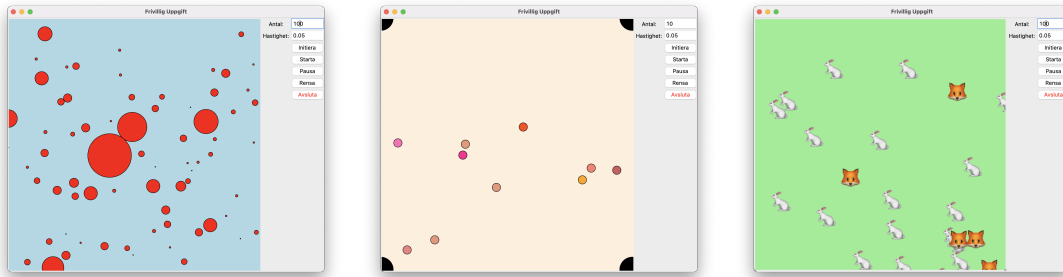
All course material that we provide is copyrighted and may not be spread.

1 Moving objects simulation

One of the two possible task options is to write a graphical user interface program where you can interact with moving objects of some kind.

You will write a program that simulates objects (e.g., balls) that move in a box. There should be a graphical user interface to specify certain properties (e.g., the number of balls and their radius), start, stop, etc.

What happens in the box is fully up to you. Some possible examples are shown below; you see the graphical interface and the sequence of events are illustrated graphically either in the same or a separate window.



We call the objects balls but they can be any type (atoms, turtles, foxes and rabbits, people, ...). The balls should have a position (a vector in the $x - y$ -plane), a velocity (also a vector in the $x - y$ -plane), and a size (e.g., a radius) and/or a type (e.g., fox/rabbit or different colors). You may add as many properties as you like.

The objects should move in the direction of the velocity vector. This velocity can be constant as long as the object does not collide with something. However, you can also change the velocity vector by introducing gravity or friction, or an element of random walk.

When there is a collision between objects and the walls of the box, you should typically change the velocity of the object by standard rules, that is, angle out is equal to angle in.

What happens when two objects collide with each other is up to you, but something should happen! Examples:

- They can bounce against each other (like billiard balls). Formula below.
- They can merge with each other to become a larger ball (like soap bubbles).
- One of the objects can remove the other object (like foxes and rabbits).
- The two objects can, sometimes, produce another object (like two rabbits become three).
- If one object is contaminated, it can transmit a disease to a non-contaminated object.

Requirement: The number of objects should change during the simulation. For example, if simulating a game of billiards one should have holes where the balls can disappear.

Objects can also have an age attribute so they can die with increased probability as they age, or develop immunity some time after having been infected.

Minimum requirements of the graphical interface

There must at least be buttons to start and stop the simulation. One can also have buttons to pause and resume (voluntary). There should at least be text fields to specify

- number of objects
- speed of the simulation (e.g., number of milliseconds between each move/frame)

The fields should have default values.

The fields should be read when clicking the start button. One should be able to start a new simulation, with other values, without restarting the program.

Graphical user interface (GUI) libraries

We advise you to use `Tkinter` to build the GUI, since it is included in all Python distributions. However, you can use any framework you wish, but it might be harder to get help.

Tkinter documentation: <https://docs.python.org/3/library/tkinter.html>

Tkinter tutorial: <https://realpython.com/python-gui-tkinter/>

Other frameworks for graphical interfaces: <https://wiki.python.org/moin/GuiProgramming>

2 Calculator web server

The other task option is to expose the calculator you have already implemented as a web server.

A web application can frequently be divided into a frontend and a backend. The frontend involves what is loaded in the web browser, which then sends various requests to the server to perform actions. Python is typically not used for frontends, although there are libraries such as FastHTML (<https://fastht.ml/>) to generate a frontend from Python code, and projects like Brython (<https://brython.info/>) to actually run Python within a web page. Typically, though, frontends are written using technologies such as HTML, CSS, and JavaScript.

You're not required to write any frontend in this task. We'll cover some examples for how to interact with a web server without a real frontend. Feel free to explore creating a frontend to your Python backend using HTML and possibly JavaScript, though.

Backend goals

You are supposed to create a small Python program running a web server. It should:

- Use some existing framework for backend Python development, such as FastAPI (<https://fastapi.tiangolo.com/>).
- Be run on a local machine with a specifiable *port number*. If the port number is 5000, the base URL for your server will be <http://127.0.0.1:5000>.
- Expose parsing of different syntax elements such as statement (corresponding to `main`, setting `ans`), `assignment`, `expression`, `factor`, `term`, reusing your logic from MA4.
- Allow the reuse of variable values between different server calls, using either cookies/a session object or a provided dictionary of known variable values in the request. If you go for the latter option, your responses should also include the current dictionary in some form. (JSON might be a format for this.) We suggest you look into cookies!
- Report errors with a specific response, preferably including an HTTP error code 400. It's not OK for the web server to crash due to invalid user input.

Making HTTP requests

A web browser uses HTTP (HyperText Transfer Protocol) to make requests for individual paths from the web server.

Since we will not have any frontend, we will need a way to test this manually, preferably

in the terminal. There are also multiple Python libraries that can be used.

These days, most operating systems include the tool curl (<https://curl.se/>), mainly developed by Daniel Stenberg, hailing from and living in Sweden. curl is highly useful for making HTTP requests in a controlled way.

One example of a hypothetical interaction, in a version using cookies and plain text for the input:

```
curl --cookie cookiefile --cookie-jar cookiefile -H 'Content-Type: text/plain' \
-d '((3+5 = a) + 4 = b) + 8' \
-X POST \
http://127.0.0.1:4998/statement
20

curl --cookie cookiefile --cookie-jar cookiefile -H 'Content-Type: text/plain' \
-d 'ans + b' \
-X POST \
http://127.0.0.1:4998/statement
32
```

Between the calls, the cookie which keeps track of your session with variable values is both stored in and retrieved from the file `cookiefile`.

Another version of the same example, using variables in JSON instead:

```
curl -H 'Content-Type: application/json' \
-d '{"data": "((3+5 = a) + 4 = b) + 8", "vars": {}}' \
-X POST \
http://127.0.0.1:4999/statement
{"value": 20, "vars": {"a": 8, "b": 12, "ans": 20}}

curl -H 'Content-Type: application/json' \
-d '{"data": "ans + b", "vars": {"a": 8, "b": 12, "ans": 20}}' \
-X POST \
http://127.0.0.1:4999/statement
{"value": 32, "vars": {"a": 8, "b": 12, "ans": 32}}
```

(The constants E and π were deliberately excluded from the output here. It's OK if your output includes them. No matter what, they should work in statements.)

Keep in mind that you'll need to handle more cases than these statement examples, and also report errors properly. In these two examples, we assume the web servers have been set up on port number 4998 and 4999, respectively. If a server fails to start on one port number, try another. Using values in the range 2000-20000 can make sense. The documentation for the framework you choose will have examples on how to start the server Python process.

Tips!

- Generally: Take small steps! Add and test one feature at a time! Use `git` to make backups!
- For the object simulation, classes like `Vector`, `Box`, `Ball`, `Fox` can be a good idea to use (and can often be tested without a GUI).
- If object i should bounce with object j , then their new velocity vectors $\mathbf{v}_i$ and $\mathbf{v}_j$ are

given by the formulas,

$$\mathbf{v}_i = \mathbf{v}_i + \frac{(\mathbf{v}_j - \mathbf{v}_i) \cdot (\mathbf{s}_j - \mathbf{s}_i)}{|\mathbf{s}_j - \mathbf{s}_i|^2}(\mathbf{s}_j - \mathbf{s}_i)$$
$$\mathbf{v}_j = \mathbf{v}_j + \frac{(\mathbf{v}_i - \mathbf{v}_j) \cdot (\mathbf{s}_j - \mathbf{s}_i)}{|\mathbf{s}_j - \mathbf{s}_i|^2}(\mathbf{s}_j - \mathbf{s}_i)$$

where $\mathbf{s}_i$ and $\mathbf{s}_j$ are the position vectors for the objects. The operation $\cdot$ is the scalar product. You can ignore the case when more than two objects collide at the same time.

- For the web server, you should most likely be able to reuse your parsing functions. You can import your MA4 solution as a module. Your web server code should supply the logic found in the `main` function in the original assignment.
- Somehow, you'll have to map server paths to Python functions that handle them. In some frameworks, this is done using function decorators.